Adaptive Weighted Sum Bi-Objective Bat for Regression Testing Optimization

Nagwa R. Fisal[1], Abeer Hamdy[2] and Essam A. Rashed[3]

[1] Suez Canal University, Egypt
[2] British University in Egypt, Egypt
[3] Nagoya Institute of Technology, Japan
Nagwa.reda7@gmail.com

Abstract. Regression testing is a type of testing carried out during the software maintenance phase, to confirm the validity of a software system after any modifications. However, regression testing is expensive, and sometimes it cannot be carried out within the testing budget, due to the large size of a test suite. In order to reduce regression testing cost, the test suite should be reduced without losing its efficiency in terms of pre-defined criteria such as its capability of fault detection; this problem is known as test suite reduction problem (TSR). In this paper, the TSR problem was formulated as a bi-objective optimization problem using an adaptive-weighted (AW) sum method. Then an adapted binary Bat algorithm (AW-ABBA) was utilized to search for a Pareto-optimal set of solutions; allowing the decision-maker under various circumstances to choose the best solution from the proposed set. The efficacy of the AW-ABBA was assessed using three metrics, Cardinality ratio, IGD^+ and Diversity, over five test suites of different sizes. Experimental results showed that the AW-ABBA was able to efficiently approximate a reference Pareto-front.

Keywords: Multi-objective, test suite reduction, binary bat, regression testing, Pareto front, NSGA-II.

1 Introduction

Software testing is used to detect faults to ensure the release of high-quality software. However, testing is expensive, so software systems are usually released with several bugs, e.g., at the early releases of Mozilla and Eclipse, about 170 and 120 bugs respectively were reported daily [1-5]. During the maintenance phase, any modification will affect one or more modules of the software, which necessitates the re-execution of previous test cases in addition to new ones. Regression testing checks the validity of the entire software under test (SUT) after each change [6]. That makes regression testing costly to imple-

ment; it accounts for over half of the total cost of testing. Test suite reduction (TSR) technique used to reduce the cost of regression testing, the size of the test suite should be reduced without affecting its efficacy, measured in terms of pre-specified criteria [7], such as mutation testing [8]. TSR problem is a multi-objective combinational optimization problem, and it is an NP-complete. Different approaches for test suite reduction have been presented in the literature [9]. Heuristic algorithms proved effectiveness in addressing each of the single objective TSR (SO-TSR) [10, 11] and multi-objective TSR (MO-TSR) problems [9, 12-17]. Recently, Yang [18, 19] proposed the Bat algorithm (BA), which is a swarm intelligence search algorithm inspired by bat echolocation behavior. Yang [18] demonstrated the superiority of BA over each of the GAs [20] and Particle swarm optimization (PSO) [21, 22] over some benchmark functions. However, for some multi-modal functions, BBA occasionally fails to find the global optimal solution. The adapted Binary Bat algorithm ABBA [7, 23] uses an updated formula to the bat velocities different from the one used by the BBA [24], which enabled ABBA to converge to better solutions than the BBA. This paper aims at solving the MO-TSR problem using the ABBA. To achieve our objectives, a fitness function was formulated using the adaptive weighted-sum approach [25], which was proved to be effective in approximating Pareto-front trade-off solutions [26] for bi-objective optimization. The performance of the adaptive weighted-sum ABBA (AW-ABBA) was evaluated in comparison to a pre-determined weighted-sum ABBA (PW-ABBA) [7], pre-determined weighted-sum GA (PW-GA) and NSGA-II. The rest of this paper is organized as follows: Section 2 summarizes previous studies that addressed the TSR problem. Section 3 explains the AW-ABBA. The experiments and results are discussed in Section 4 finally, section 5 concludes the paper and introduces future directions for this research.

2 Literature Review

Researchers proposed a variety of approaches for reducing the test suite size. Most of these approaches depend on greedy algorithms [27], with only a few depending on clustering algorithms [28] or hybrid algorithms [29]. Recently, heuristic algorithms have been used to solve single and multi-objective TSR [9, 11-17]. As mentioned by Khan et al. [9], the most used heuristics search-based for TSR problem are single-objective optimization,

which represents 79% of them. Different studies have shown that some heuristic algorithms outperform greedy algorithms in solving the single and multi-objective TSR problems [11, 14]. The work of Yoo and Harman [14] is considered the first multi-objective optimization study to reduce the size of the test suite. The authors employed the NSGA-II algorithm and demonstrated its superiority over greedy techniques in an experiment. The NSGA-II method was also used by Geng et al. [12] and Gupta et al. [17]. Wang et al. [15] used different forms of weighted-sum method GA; their experiments showed that random weighted-sum GA outperformed NSGA-II and other common multi-objective algorithms. Wei et al. [16] compared six multi-objective optimization algorithms, including NSGA-II and several variants of the multi-objective decomposition-based evolutionary algorithm (MOEA/D). NSGA-II outperformed the MOEA/D in small programs, whereas the MOEA/D outperformed the NSGA-II in large programs. The approach proposed in this paper is a heuristic search-based approach, that utilizes the ABBA [23].

3 Adaptive Weighted Sum Bi-Objective ABBA for TSR

3.1 TSR Solution Encoding

Each TSR solution is represented as bat's position vector X_i, i.e., each X_i represents a reduced test suite. X_i is encoded as a binary vector $X_i = (x_{i1}, x_{i2}, \ldots, x_{id})$, where, d is the total number of test cases in the original test suite TS, each bit x_{ij} corresponds to a test case tc_j, the value of bit is equal to "1" or "0". This means that tc_j is included/excluded in the reduced test suite, respectively.

3.2 TSR Bi-Objective Fitness Function Formulation

Fitness function was formulated using two objectives, defined by Eq. (1). The first objective is to lower the test suite's cost. The second objective aims at maximizing the number of faults detected by a reduced test suite. The mutation score is used to express the reduced test suite's fault detection capability.

$$Objective1 = \frac{\sum_{|TS|} exec_time_i}{\sum_{|RTS|} exec_time_i} \quad , \qquad Objective2 = \frac{|killed\ mu|}{|mu|} \tag{1}$$

where $|TS|$ is the original test suite size, $|RTS|$ is the reduced test suite size, $exec_time_i$ is the execution time of tc_i, $|mu|$ is the total number of mutants and $|killed\ mu|$ is the number of mutants killed by the reduced test suite RTS.

To formulate the fitness function, we used the adaptive weighted-sum method for bi-objective optimization [25]; as defined by Eq. (2), the best solution is the one that maximizes the $fitness$.

$$fitness = (1-\lambda) * nor(Objective1) + \lambda * nor(Objective2) \quad (2)$$

$$nor(x) = \frac{x}{x+1} \quad (3)$$

Where, $nor(x)$ is a normalization function [30], $\lambda \in [0,1]$, is a weighting factor that sets the relative importance of each objective. λ is not determined previously, it evolves according to the shape of the Pareto front of the problem.

The Adaptive weighted-sum method starts by applying the normal weighted-sum method, with a large weight step size, $\Delta\lambda$, to generate coarse representations of the solutions that form the Pareto-front. The algorithm proceeds according to the length of each segment (distance between neighboring solutions) such that closely overlapping solutions are deleted, while long-distance segments are refined by constructing additional inequality constraints in this region and performing sub-optimization with the weighted-sum method. When all the segments of the Pareto front reach a pre-specified resolution, the algorithm ends. The detailed procedure of the adaptive weighted-sum method is as follows.

Step 1: Use the normal weighted-sum approach with a small number of divisions, $n_{initial}$. The number of divisions determines the uniform step size $\Delta\lambda$ of the weighting factor λ as defined by Eq. (4).

$$\Delta\lambda = \frac{1}{n_{initial}} \quad (4)$$

Step 2: Calculate the segment lengths between all neighboring solutions. Delete solutions that are closely overlapping. When using the weighted-sum method, it is common to get several almost identical solutions. The Euclidian distances among these solutions are nearly zero, and the Pareto front can be represented by only one of them. So, if the distances among a set of solutions are smaller than a pre-specified distance value (ε), all these solutions are discarded except one.

Step 3: Determine how many more refinements each segment requires. The refinements are determined according to the relative lengths of the segments by Eq. (5).

$$n_i = round(\, c\,\frac{l_i}{l_{avg}}) \quad \text{for the } i^{\text{th}} \text{ segment} \tag{5}$$

where n_i is the number of further refinements for the ith segment, l_i denotes the ith segment length, l_{avg} is the average length of all the segments, and c denotes an algorithmic constant. 'Round' is a function that rounds to the nearest integer. If $n_i \leq 1$ the segment is not refined any further. If $n_i > 1$ for any segment i proceed to the next step for further refinement.

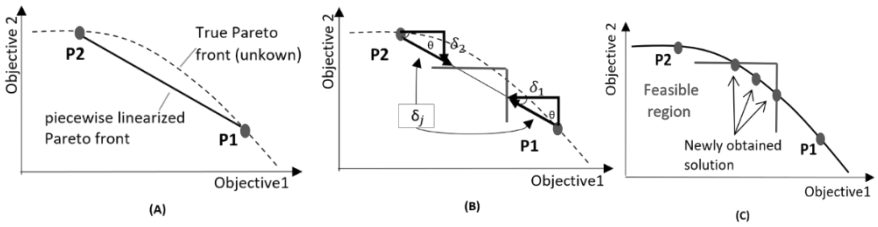


Fig. 1. Determining the offset distances, *δ1* and *δ2*, based on δ_J

Step 4: Calculate the offset distances δ_1, δ_2 from the two endpoints of each segment. Firstly, link the endpoints, P1 and P2, to form a piecewise linearized secant line; as shown in Fig. 1 (a). Then, the user selects the offset distance along the piecewise linearized Pareto front, δ_J (Fig. 1 (b)); δ_J determines the density of the Pareto solution distribution. The angle θ in Fig. 1 (b) is calculated using Eq. (6) to find the offset distances parallel to the objective axes.

$$\theta = tan^{-1}(-\frac{p_1^y - p_2^y}{p_1^x - p_2^x}) \tag{6}$$

where p_i^x and p_i^y are the x(Objective1) and y(Objective2) positions of the end points, P1 and P2, respectively. Then, δ1 and δ2 are determined using δ_J and θ as follows:

$$\delta_1 = \delta_J \cos\theta \quad and \quad \delta_2 = \delta_J \sin\theta \tag{7}$$

Step 5: In each of the feasible regions, construct additional inequality constraints and conduct sub-optimization with the weighted-sum method. As shown in Fig. 1(b,c), the feasible region is offset from P1 and P2 in the direction of Objective1 and Objective2 by the distances of δ_1 and δ_2. Performing suboptimization in this region, the problem is formulated as:

$$\text{Max} \quad (1-\lambda) * nor(Objective1) + \lambda * nor(Objective2) \tag{8}$$
$$s.t. \quad Objective1 \leq p_1^x - \delta_1 \ , \quad Objective2 \leq p_2^y - \delta_2$$

where δ_1 and δ_2 are the offset distances, p_1^x and p_2^y are the x and y positions of the endpoints. The uniform step size of the weighting factor λ for each feasible region is determined using Eq. (9):

$$\Delta\lambda_i = \frac{1}{n_i} \tag{9}$$

Step 6: Calculate the segment lengths between all neighboring solutions. Delete solutions that are almost overlapping. Terminate the process if the length of all segments is less than δ_J. Otherwise, go to Step 4 and iterate if there are segments in which their lengths are greater than the maximum length δ_J.

Four important parameters that must be set: the number of the Pareto front segments in the initial iteration ($n_{initial}$); the offset distance (δ_J); the Euclidean distance for specifying the nearly overlapping solutions (ε); the further refinement constant (c). Table 1 lists the parameter settings we experimented with.

Table 1. Parameter settings for Adaptive weighted sum method

Parameter	δ_J	ε	c	$n_{initial}$
Value	0.1	0.25 * δ_J	3	15

4 Experiments and Results

4.1 Data Sets

We experimented with five Java programs provided by Polo et al. [31]; Table 2, lists their characteristics which are: lines of code (LOC), test suite size |*tc*|, number of mutants |*mu*|, and execution time of the test suite (T_{exe}).

Table 2. Software under test (SUT).

	MBisectOk	MBubCorrect	MFindOk	MMidOK	MTringle
LOC	31	54	79	59	61
|*mu*|	44	70	179	138	239
|*tc*|	25	256	135	125	216
T_{exe}	25	256	135	125	216

4.2 Performance Metrics

Three metrics were used which are: Cardinality, Diversity and Inverted Generational Distance plus, to measure the convergence, diversity (spread and uniformity),

and cardinality of the approximated solution set to the known Pareto-optimal front [32]. Assume, A is an approximated Pareto front and R is a reference Pareto front, which includes all non-dominated solutions that are already known. And m is the dimension of the objective space; the metrics are defined as follows:

(1) Cardinality Ratio (CR): The cardinality of a set A, $|A|$, can be defined as the number of NDS in this set. Cardinality Ratio of a set A, $CR(A)$, is the ratio between $|A|$ and the cardinality of R ,$|R|$, as given by Eq. (9)

$$Cardinality\ Ratio\ CR(A) = \frac{|A|}{|R|} \tag{9}$$

(2) Inverted Generational Distance plus (IGD^+): IGD^+ [33] used to measure the convergence of A. It measures the distance to the reference Pareto front R using a distance measure that adopts Pareto dominance as given by Eq. (10, 11).

$$IGD^+(A,R) = \frac{1}{|R|}\sum_{\vec{r}\in R} min_{\vec{a}\in A}\left(d^+(\vec{a},\vec{r})\right) \tag{10}$$

where, $$d^+(\vec{a},\vec{r}) = \sqrt{\sum_{k=1}^{m}(\max\{r_k - a_k\ ,0\})^2} \tag{11}$$

(3) Diversity (Δ): It evaluates the spread and distribution of solutions on the known Pareto-optimal front. The spread refers to the range of values covered by the solutions in A, while the distribution is the relative distance among solutions in A. It's calculated by Eq. (12).

$$\Delta = \frac{d_f + d_l + \sum_{i=1}^{|A|-1}|d_i - \bar{d}|}{d_f + d_l + (|A|-1)\bar{d}} \tag{12}$$

where d_i is the Euclidean distance between two consecutive solutions in A, d_f and d_l are the distances between the extreme solutions on the refernce Pareto front R and the boundary solutions of A. $\bar{d}$ is the average of all distances $d_i, i \in [1, |p| - 1]$. The value of Δ will be in the range of 0 to 1.

As R is a reference Pareto front (the true front is unknown), the Δ value of A should be compared with the Δ of R. So, in our work Δ of A computed as Eq. (13).

$$\Delta = \Delta_R - \Delta_A \tag{13}$$

Pareto fronts with a higher cardinality ratio, smaller IGD^+ and smaller spread are preferred.

4.3 Results

A set of experiments were conducted on the five Java SUT. In all the experiments, the maximum number of iterations was set to 100. Table 3 shows the predetermined values of λ that were used to generate ten non-dominated solutions (NDS) by the PW-ABBA and PW-GA.

Table 3. values of λ used with the PW-ABBA and PW-GA

NDS	1	2	3	4	5	6	7	8	9	10
λ	0.6	0.676	0.744	0.804	0.856	0.9	0.936	0.964	0.984	0.9960

As could be observed from table 4, NSGA-II has the largest CR over the five programs, which means that NSGAII was capable of discovering the largest number of NDS. However, CR is not enough to prove the superiority of a heuristic algorithm as the quality of the discovered NDS should be analyzed to determine their closeness to the true Pareto front (in terms of IGD+) and their diversity. As it could be observed, the performance of AW-ABBA is superior to the other algorithms, in terms of the IGD+, over three programs (MBisectOk, MMidOK and MTringle); where their IGD+ values are equal or close to zero, which means that the NDS discovered by the AW-ABBA are located on or very close to the reference front. Over Mfindok program, the solutions discovered by each of the PW_ABBA, AW-ABBA and NSGA-II are on the reference Pareto-front (IGD+=0). It could be observed from table 6 that the diversity values of PW_ABBA and AW_ABBA are close to each other and superior to the NSGA-II over three programs. Finally, PW-GA has the worst performance results among all the algorithms; it approximates the pareto-front with the largest IGD+ and diversity values, i.e., the discovered NDS are located far from the reference front and their distribution and spread are not efficient. Fig.2 illustrates samples of approximate Pareto fronts discovered by the different algorithms. For MMid and Mtringle, whose reference fronts include the largest number of NDS, the figures show the efficiency of AW-ABBA over other algorithms in approximating the Pareto front.

Table 4. CR of the approximated Pareto-fronts

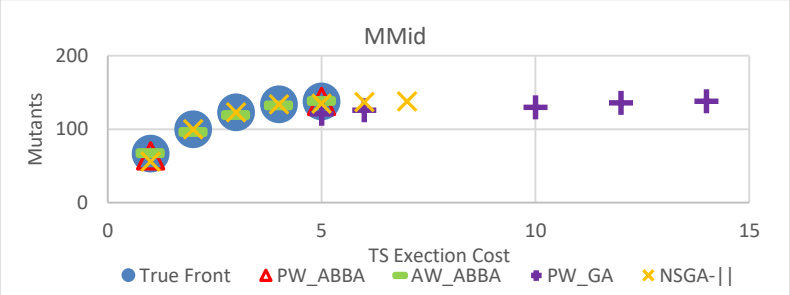
	PW-ABBA	AW-ABBA	GA	NSGA-II
MBisectOk	2/2	1/2	1/2	**3/2**
MBubCorrect	1/3	2/3	1/3	**3/3**
MFindOk	1	1	1	1
MMidOK	2/5	5/5	5/5	**7/5**
MTringle	3/16	16/16	2/16	**17/16**

Table 5. IGD^+ of the approximated Pareto-fronts

	PW-ABBA	AW_ABBA	GA	NSGA-II
MBisectOk	**0**	**0**	0.6	0.08
MBubCorrect	0.4	0.2	0.5	**0.08**
MFindOk	0	0	0	0
MMidOK	0.25	**0.06**	0.3	0.1
MTringle	0.1	**0.00**	0.1	0.1

Table 6. Diversity of the approximated Pareto fronts

	PW-ABBA	AW-ABBA	GA	NSGA-II
MBisectOk	**0**	**0**	1.00	-0.26
MBubCorrect	1.00	**0.32**	1.00	-0.26
MFindOk	1	1	1	1
MMidOK	0.53	**0.11**	0.24	0.38
MTringle	**0.01**	0.02	0.01	-0.003



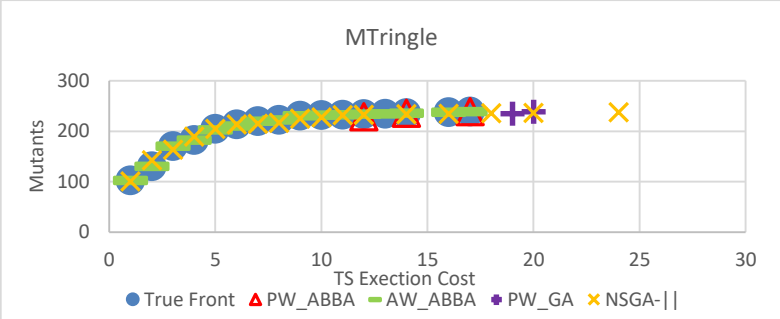


Fig.2. Sample of the approximated NDS set of TSR by different algorithms.

5 Conclusion and Future Work

This paper proposed solving the test suite reduction problem, as a bi-objective optimization problem using the ABBA; which is one variant of the BBA and was reported to be more effective than the BBA and the PSO. An adaptive weighted-sum method was utilized to formulate the fitness function. The effectiveness of the adaptive weighted-sum ABBA (AW-ABBA) was evaluated over five Java programs and compared to a pre-determined weighted-sum ABBA(PW-ABBA), PW-GA and NSGA-II. The experimental results showed that the performance of the proposed AW-ABBA is superior to PW-ABBA and PW-GA. Although the NSGA-II could discover more non-dominated solutions than each of the AW-ABBA, PW-ABBA; the NDS discovered by AW-ABBA, over most of the programs, dominate some of the NDS discovered by the NSGA-II.

As a further extension for this work, more objectives such as branch coverage could be considered in the fitness function.

REFERENCES

[1] A. Hamdy and A. Ellaithy, "Multi-Feature approach for Bug severity assignment," *International Journal of Open Source Software Processes,* vol. 11, no. 2, pp. 1-15, 2020.

[2] A. Hamdy and A. El-Laithy, "Smote and feature selection for more effective bug severity prediction," *International Journal of Software Engineering Knowledge Engineering,* vol. 29, no. 06, pp. 897-919, 2019.

[3] A. Hamdy and A. El-Laithy, "Semantic categorization of software bug repositories for severity assignment automation," in *Integrating Research and Practice in Software Engineering*: Springer, 2020, pp. 15-30.

[4] A. Hamdy and G. Ezzat, "Deep mining of open source software bug repositories," *International Journal of Computers Applications,* pp. 1-9, 2020.

[5] A. Hamdy *et al.*, "Deep Hybrid Features for Code Smells Detection," *Journal of Theoretical Applied Information Technology,* vol. 98, no. 14, 2020.

[6] R. H. Rosero, O. S. Gómez, and G. Rodríguez, "15 years of software regression testing techniques—a survey," *International Journal of Software Engineering Knowledge Engineering,* vol. 26, no. 05, pp. 675-689, 2016.

[7] N. Reda, A. Hamdy, and E.-A. Rashed, "Multi-Objective Adapted Binary Bat for Test Suite Reduction," vol. 31, no. 2, pp. 781--797, 2022.

[8] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE transactions on software engineering,* vol. 37, no. 5, pp. 649-678, 2010.

[9] S. U. R. Khan, S. P. Lee, N. Javaid, and W. Abdul, "A systematic review on test suite reduction: Approaches, experiment's quality evaluation, and guidelines," *IEEE Access,* vol. 6, pp. 11816-11841, 2018.

[10] S. K. Mohapatra, A. K. Mishra, and S. Prasad, "Intelligent Local Search for Test Case Minimization," *Journal of The Institution of Engineers (India): Series B,* vol. 101, no. 5, pp. 585-595, 2020/10/01 2020.

[11] J.-z. Sun and S.-y. Wang, "A novel chaos discrete particle swarm optimization algorithm for test suite reduction," in *The 2nd International Conference on Information Science and Engineering*, Hangzhou, China, 2010, pp. 1-4: IEEE.

[12] J. Geng, Z. Li, R. Zhao, and J. Guo, "Search based test suite minimization for fault detection and localization: A co-driven method," in *International Symposium on Search Based Software Engineering*, 2016, pp. 34-48: Springer.

[13] S. Mohanty, S. K. Mohapatra, and S. F. Meko, "Ant colony optimization (ACO-Min) algorithm for test suite minimization," in *Progress in Computing, Analytics and Networking*: Springer, 2020, pp. 55-63.

[14] S. Yoo and M. Harman, "Pareto efficient multi-objective test case selection," in *Proceedings of the 2007 international symposium on Software testing and analysis*, New York, NY, USA, 2007, pp. 140-150.

[15] S. Wang, S. Ali, and A. Gotlieb, "Cost-effective test suite minimization in product lines using search techniques," *Journal of Systems Software,* vol. 103, pp. 370-391, 2015.

[16] Z. Wei, W. Xiaoxue, Y. Xibing, C. Shichao, L. Wenxin, and L. Jun, "Test suite minimization with mutation testing-based many-objective evolutionary optimization," in *2017 International Conference on Software Analysis, Testing and Evolution (SATE)*, Harbin, China, 2017, pp. 30-36: IEEE.

[17] N. Gupta, A. Sharma, and M. K. Pachariya, "Multi-objective test suite optimization for detection and localization of software faults," *Journal of King Saud University-Computer Information Sciences,* 2020.

[18] X.-S. Yang, "A new metaheuristic bat-inspired algorithm," in *Nature inspired cooperative strategies for optimization (NICSO 2010)*: Springer, 2010, pp. 65-74.

[19] X.-S. Yang, "Bat algorithm for multi-objective optimisation," *International Journal of Bio-Inspired Computation,* vol. 3, no. 5, pp. 267-274, 2011.

[20] A. Hamdy, "Genetic fuzzy system for enhancing software estimation models," *International Journal of Modeling Optimization,* vol. 4, no. 3, p. 227, 2014.

[21] A. Hamdy and A. Mohamed, "Greedy binary particle swarm optimization for multi-objective constrained next release problem," *International Journal of Machine Learning Computing,* vol. 9, no. 5, pp. 561-568, 2019.

[22] R. Eberhart and J. Kennedy, "A new optimizer using particle swarm theory," in *MHS'95. Proceedings of the Sixth International Symposium on Micro Machine and Human Science*, Nagoya, Japan, 1995, pp. 39-43: IEEE.

[23] N. R. Fisal, A. Hamdy, and E. A. Rashed, "Search-Based Regression Testing Optimization," *International Journal of Open Source Software Processes,* vol. 12, no. 2, pp. 1-20, 2021.

[24] S. Mirjalili, S. M. Mirjalili, and X.-S. Yang, "Binary bat algorithm," *Neural Computing Applications,* vol. 25, no. 3-4, pp. 663-681, 2014.

[25] I. Y. Kim and O. L. De Weck, "Adaptive weighted-sum method for bi-objective optimization: Pareto front generation," *Structural multidisciplinary optimization,* vol. 29, no. 2, pp. 149-158, 2005.

[26] P. Ngatchou, A. Zarei, and A. El-Sharkawi, "Pareto multi objective optimization," in *Proceedings of the 13th International Conference on, Intelligent Systems Application to Power Systems*, Arlington, VA, USA, 2005, pp. 84-91: IEEE.

[27] R. A. Assi, W. Masri, and C. Trad, "Substate Profiling for Effective Test Suite Reduction," in *2018 IEEE 29th International Symposium on Software Reliability Engineering (ISSRE)*, Memphis, TN, USA, 2018, pp. 123-134: IEEE.

[28] C. Coviello, S. Romano, G. Scanniello, A. Marchetto, G. Antoniol, and A. Corazza, "Clustering support for inadequate test suite reduction," in *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, Campobasso, Italy, 2018, pp. 95-105: IEEE.

[29] Z. Anwar, H. Afzal, N. Bibi, H. Abbas, A. Mohsin, and O. Arif, "A hybrid-adaptive neuro-fuzzy inference system for multi-objective regression test suites optimization," *Neural Computing Applications,* vol. 31, no. 11, pp. 7287-7301, 2019.

[30] D. Greer and G. Ruhe, "Software release planning: an evolutionary and iterative approach," *Information software technology,* vol. 46, no. 4, pp. 243-253, 2004.

[31] M. Polo, M. Piattini, and I. García-Rodríguez, "Decreasing the cost of mutation testing with second-order mutants," *Software Testing, Verification Reliability,* vol. 19, no. 2, pp. 111-131, 2009.

[32] J. G. Falcón-Cardona and C. A. C. Coello, "Indicator-based multi-objective evolutionary algorithms: A comprehensive survey," *ACM Computing Surveys,* vol. 53, no. 2, pp. 1-35, 2020.

[33] H. Ishibuchi, H. Masuda, Y. Tanigaki, and Y. Nojima, "Modified distance calculation in generational distance and inverted generational distance," in *International conference on evolutionary multi-criterion optimization*, 2015, pp. 110-125: Springer.

View publication stats